BMU Website Chatbot

An agentic RAG assistant that answers university questions from official BMU pages, with guardrails and full tracing.

WHY

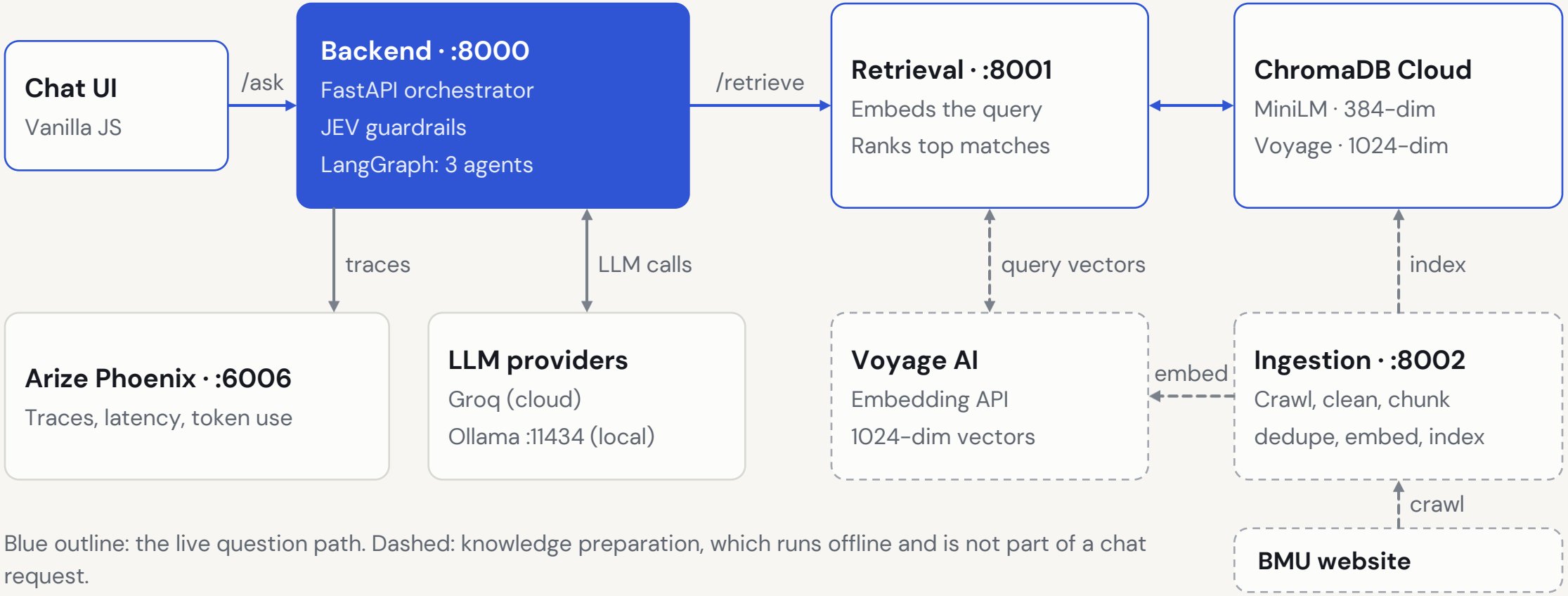
A plain LLM is not enough

Problem	How the chatbot handles it
Answers are spread over many pages	RAG search over 1,258 indexed BMU page summaries
LLMs can invent fees, dates and cut-offs	A Context Auditor checks the evidence before any answer
Prompt injection and off-topic questions	JEV guardrails on both input and output
Slow or wrong answers are hard to debug	Arize Phoenix traces every step

Retrieve first, check the evidence, then answer.

ARCHITECTURE

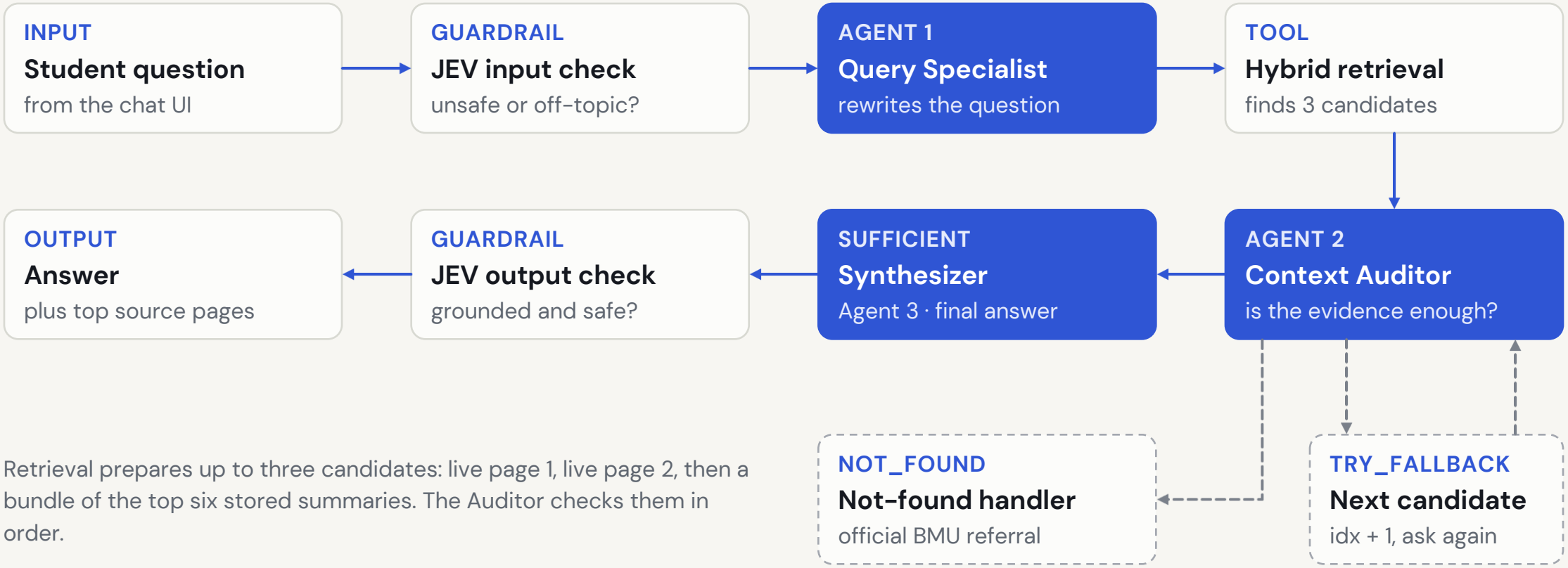
Four services around one backend



Blue outline: the live question path. Dashed: knowledge preparation, which runs offline and is not part of a chat request.

WORKFLOW

How one question is answered



Retrieval prepares up to three candidates: live page 1, live page 2, then a bundle of the top six stored summaries. The Auditor checks them in order.

AGENTS

Three agents and one retrieval tool

AGENT 1

Query Specialist

Turns a casual question into a precise search query with keywords and intent.

Groq · gpt-oss-20b

TOOL

Hybrid retrieval

Finds the closest BMU pages. Score = 0.68 × meaning + 0.32 × keywords.

No LLM · port 8001

AGENT 2

Context Auditor

Reads one candidate at a time and decides: SUFFICIENT, TRY_FALLBACK or NOT_FOUND.

Groq · gpt-oss-20b

AGENT 3

Response Synthesizer

Writes the final answer using only the approved context. No guessing.

Groq · gpt-oss-120b

Local mode: all three agents can run on Ollama (llama3.2:3b) instead. A switch in the chat UI picks Groq or Local.

LLM PROVIDERS

Groq cloud or local Ollama

Dimension	Groq (cloud)	Local (Ollama)
Where it runs	Cloud-hosted	On your own machine
Model	GPT-OSS models	Llama 3.2 3B
Latency (reported)	0.85–2.1 s per agent	15–30 s on CPU
Network	Required	Not needed after setup
Privacy	Requests go to the cloud API	Data can stay local
Cost	API plan and token usage	No API fee, uses your hardware

Groq is faster in the reports. Ollama suits offline use and local data control. Next step: run the same 100 questions on both and compare.

GUARDRAILS

JEV checks both sides of the agents

BEFORE THE AGENTS

Input guardrail

Prompt injection
12 patterns, blocked at 0.70 or higher

Toxicity
4 patterns, blocked at 0.70 or higher

Scope
BMU and academic questions pass, off-topic ones are blocked

AFTER THE AGENTS

Output guardrail

Grounding
Answer words must overlap the context (score 0.40 or higher)

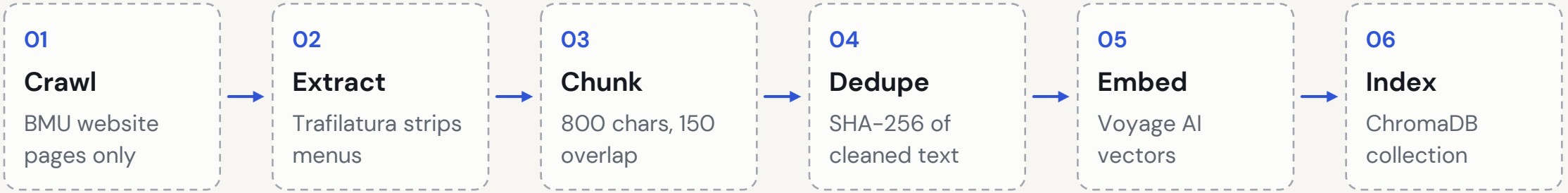
Weak grounding
A note is added: verify on the official BMU portal

Safety
Unsafe answers are withheld

Runs locally in milliseconds. The cloud TypeSafe check is optional and not enabled yet. The UI switch that turns guardrails off is for testing only.

KNOWLEDGE

From BMU pages to searchable vectors



1,258

page summaries in the production collection

~16×

text compression, about 40,000 raw lines down to 2,500

384 · 1024

MiniLM and Voyage vector sizes. Never mix them in one search.

SUMMARIZER

Why the page summarizer is hybrid

01
TextRank
Broke on tables and lists

02
TF-IDF
Repeated common words

03
One sentence
Too little information

04
Many sentences
Near-duplicate sentences

05 · Final
Hybrid v5
Rank, drop duplicates, keep 3

The final recipe

Split sentences, encode them (384-dim), rank against the page centroid, drop sentences more than 0.75 similar to one already chosen, keep 3, then compress. About 16× smaller, with no GPU and no generated text.

SERVICES

Who does what

Service	Port	File	Job
Backend	8000	semantic.py	Runs /ask, guardrails and the agent graph
Retrieval	8001	retrieval_service.py	Embeds the query and ranks ChromaDB hits
Ingestion	8002	ingestion_service.py	Crawls, chunks, dedupes and indexes pages
Phoenix	6006	phoenix_service.py	Shows traces, latency and token use
Ollama (optional)	11434	none, external runtime	Local LLM (llama3.2:3b)

Why separate services? Heavy embedding, slow crawling and tracing stay out of the chat path, and each service can be tested alone.

OBSERVABILITY

See every step in Phoenix

One trace per question

bmu_rag_pipeline

- LangGraph run
  - query_specialist
  - retrieval_tool
  - context_auditor (once per candidate)
  - response_synthesizer
- LLM calls, one per agent step

What it records

Total and per-agent latency

Provider and model: Groq or Local

Guardrail status and action

Token counts for each LLM call

Input and output text

Phoenix runs on port 6006. If it is down, the chatbot still answers, just without traces. Prompts are captured, so keep API keys out of them.

BENCHMARK

100 questions, 20 categories

01 Admissions	02 Fees	03 Scholarships	04 B.Tech CSE
05 MBA	06 Law	07 Liberal studies	08 Campus life
09 Hostel	10 Sports	11 Library	12 Placements
13 Internships	14 Faculty, research	15 Academic policies	16 Location, transport
17 Health, amenities	18 Complex questions	19 Out-of-scope	20 Prompt injection

5 questions per category make 100 test cases. The plan is to run every question through both Groq and Local, so the comparison is fair. The blue chips test the guardrails.

METRICS

How answers are scored

Metric	Question it answers	Weight
Relevance	Does the answer address the question?	25%
Faithfulness	Are the claims supported by the retrieved context?	25%
Correctness	Does it match the verified reference answer?	25%
Completeness	Are the required facts included?	15%
Conciseness	Is it brief, without repetition?	10%

Guardrails

Attacks blocked, legitimate questions passed

Latency

Median and p95 time per provider

Reliability

Errors, timeouts and rate-limit retries

EVALUATION RUN

Planned run: Groq against Local

01
Dataset
100 questions with reference answers

→

02
Dry run
Check setup before spending quota

→

03
Paced runner
Spaces requests, backs off on 429

→

04
Checkpoint
Saves each result, resumes later

→

05
Dashboard
Side-by-side charts, CSV and JSON export

Groq rate limits

The planned 1.8 s delay is too short. With about 2,500 tokens per question and an 8,000 tokens/min limit, only about 3 questions fit per minute: one every 19 seconds.

Fair comparison

Same 100 questions through Groq and Local (Ollama, llama3.2:3b). For each one we record the answer, the context, the guardrail decision and the timing.

RESULTS

100-question evaluation results

1.95 / 5 Overall quality score	**1.49 / 5** Faithfulness	**2.38 / 5** Relevance
3.47 / 5 Conciseness	**57%** Retrieval hit rate	**95%** Guardrail accuracy
10.76 s Median end-to-end latency	**23.87 s** P95 latency	**0 / 100** Evaluation errors

Reported results from the 100-question run across 20 categories.

FINDINGS

What the results tell us

Strong

Security and prompt-injection handling

All 100 questions completed without errors

Answers are concise (3.47 / 5)

Needs work

Retrieval coverage: hit rate is 57%

Faithfulness: 1.49 / 5

Knowledge gaps: health and amenities, detailed CSE syllabi, scholarships

Scores use proxy metrics, so treat them as indicators, not proof. The fix is mostly better knowledge coverage and retrieval.

DASHBOARD

Evaluation dashboard

Overview
Cases completed, overall scores, error rate

Provider comparison
Groq and Local on all five quality metrics

Latency
Median and p95 charts for each provider

Guardrail analysis
Wrongly blocked and wrongly passed questions

Question explorer
Filter by category, difficulty and provider

Side by side
Reference, Groq and Local answers with sources

Designed with Google Stitch MCP (theme: Obsidian Telemetry), with CSV and JSON export. Status: design stage. It must read real saved results before we trust it.

FUTURE SCOPE

What we will do next

01

Expand the knowledge base

Index BMU handbooks, course syllabi, scholarship policies and fee documents

02

Improve retrieval accuracy

Optimize hybrid search, query rewriting, chunking and ranking

03

Strengthen security

Expand adversarial testing, improve injection detection, check false positives and negatives

04

Enhance evaluation

Run the same 100 questions on Groq and Ollama and compare quality, latency and reliability

05

Evaluation dashboard

Connect it to real benchmark results, with category analysis and CSV/JSON export

06

Continuous monitoring

Use Phoenix tracing and automated benchmark runs to catch regressions after every update

Thank you

Questions are welcome.